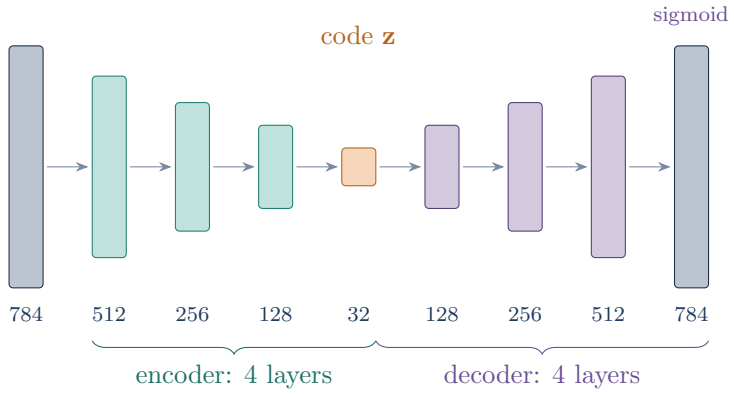
Your first autoencoder

step by step, on the handwritten digit 5

A guidebook for the lab session



What you will build, in order

1. a function that loads only the images of the digit **5** from MNIST, and one that displays them;
2. the **DataLoaders** that serve the images in mini-batches;
3. a **vanilla autoencoder**: 4 linear layers to encode, 4 to decode, a sigmoid at the end;
4. a **training function**, with the reconstructions displayed after **every epoch**;
5. the **loss curves**;
6. finally, the same autoencoder rebuilt with **convolutions**.

Contents

0 The big picture **2**

1 Load the digit 5 and look at it **2**

1.1 Loading . . . 3

1.2 Displaying . . . 3

2 The DataLoaders **4**

3 The autoencoder **5**

4 One epoch of training, and evaluation **6**

5 Showing the reconstructions **8**

6 The full training loop **9**

7 The loss curves **10**

8 From linear layers to convolutions **12**

8.1 What changes, what stays . . . 12

8.2 The shape arithmetic . . . 12

8.3 Train it and compare . . . 13

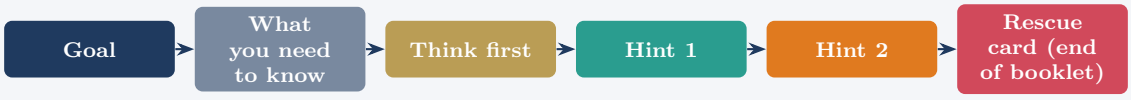
A Rescue cards **15**

B Answers to the "Think first" calculations **17**

C Error messages and what they mean **18**

How to use this guidebook

You work in `05_vanilla_autoencoder_starter.ipynb`. Each step of this booklet matches one section of the notebook, and each section ends with a **check cell**. Run it: when it prints `Step k OK`, move on.
Every step is organised the same way, from the most open to the most guided:



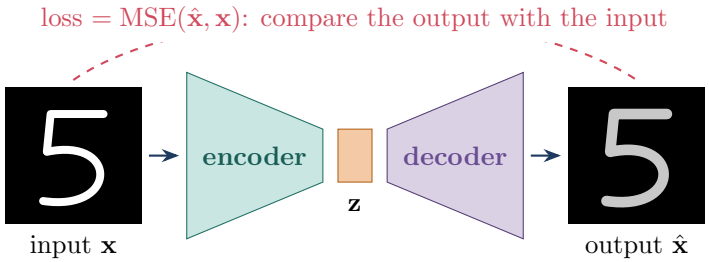
The rule of the game: try to write the code after the *Think first* box. Read a hint only after a real attempt, and only one at a time. The rescue cards at the back are code skeletons with a few blanks; the complete code is in the solution notebook, which you open only at the very end, to compare.

Three debugging reflexes

- **Print shapes.** `print(x.shape)` after each operation solves most problems in deep learning.
- **Print ranges and types.** `x.min()`, `x.max()`, `x.dtype`: images must be `float32` in $[0, 1]$.
- **Read the last line of the error first.** It usually names the two shapes, or the two devices, that disagree.

Step 0. The big picture

An **autoencoder** is a network trained to output its own input. That sounds useless, until you force the information through a narrow middle layer, the *code* $\mathbf{z}$. To rebuild a $28 \times 28 = 784$-pixel image from only 32 numbers, the network must discover what all the images have in common, and what makes each of them different.



What you need to know

- There are **no labels** in this lab. The image is both the input and the target.
- We train on a single digit, the **5**. The autoencoder only has to learn the ways people write a 5: slant, size of the loop, length of the top bar, thickness of the stroke.
- The last layer is a **sigmoid**, $\sigma(t) = 1/(1 + e^{-t})$, which outputs numbers in $(0, 1)$, exactly the range of the pixels.
- The loss is the **mean squared error** between the output and the input pixels.

Think first (answer on paper before coding)

1. The code has 32 numbers and the image has 784. By how much is the image compressed?
2. If the code had 784 numbers, there would be a trivial perfect solution. Which one? Why is it useless?
3. Why is the sigmoid a natural choice for the last layer here, and why would it be a bad choice for the code $\mathbf{z}$?

Step 1. Load the digit 5 and look at it

Goal of this step

Write `load_digit(digit, train)`, which returns a `float32` tensor of shape $(N, 1, 28, 28)$ with values in $[0, 1]$ containing **only** the images of `digit`. Then write `show_digits(images, n, title)`, which displays the first `n` images side by side.

1.1 Loading

What you need to know

- `torchvision.datasets.MNIST(root="data", train=True, download=True)` downloads MNIST (once) and returns a dataset object. The first line of the function is already written for you.
- That object stores *all* 60 000 training images in one tensor, `dataset.data`, of shape $(60000, 28, 28)$ and type `uint8` (integers from 0 to 255), and all the labels in `dataset.targets`, of shape $(60000,)$.
- Comparing a tensor with a number, as in `t == 3`, gives a **boolean tensor** of the same shape, `True` where the condition holds.
- A boolean tensor can be used as an **index**: `t[mask]` keeps only the entries where `mask` is `True`.
- PyTorch convolution and plotting code expects images as (batch, channels, height, width). MNIST is grey-level: one channel.

Think first (answer on paper before coding)

1. Write down the shape of the data at each stage: all images, only the 5s, after converting to $[0, 1]$, after adding the channel.
2. Why divide by 255, and why must the result be a float?
3. Try in a scratch cell: `t = torch.tensor([3, 5, 7, 5]); print(t == 5, t[t == 5])`.

Hint 1 (a nudge)

Four short lines are enough: build a mask from the labels, use it to select images, convert them to floats in $[0, 1]$, add a dimension of size 1 at position 1.

Hint 2 (the tools)

`dataset.targets == digit`, `dataset.data[mask]`, `.float() / 255.0`, `.unsqueeze(1)`.

1.2 Displaying

What you need to know

- `fig, axes = plt.subplots(1, n, figsize=(1.5 * n, 1.8))` creates one row of `n` small plots; `axes[i]` is the i-th one.
- `axes[i].imshow(img, cmap="gray", vmin=0, vmax=1)` draws a 2-D array in grey levels. Fixing `vmin` and `vmax` prevents matplotlib from rescaling the contrast of each image differently.
- `imshow` wants a 2-D array of shape $(28, 28)$, not $(1, 28, 28)$.

Hint 1 (a nudge)

Loop over `i` from 0 to `n-1`; for each, draw image `i` of channel 0 in `axes[i]` and hide its axes. Put the title on the whole figure, then show it.

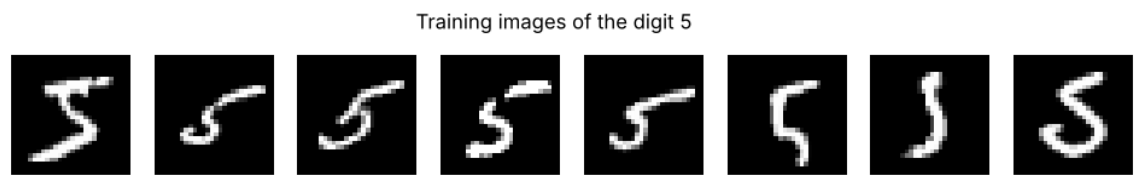
Hint 2 (the tools)

`images[i, 0]` is the 28×28 image. Add `.detach().cpu()` so that the same function works later for tensors produced by the network on a GPU. Then `axes[i].axis("off")`, `fig.suptitle(title)`, `plt.show()`.

Still stuck after both hints? Use rescue card 1 at the end of this booklet (page 15).

Checkpoint (you are on track if…)

The check cell prints `train: (5421, 1, 28, 28) test: (892, 1, 28, 28)` and draws a row of 5s like this one:



Notice how different they are: slanted, upright, thin, thick, open or closed loop. This variety is what the code **z** will have to capture.

Common pitfalls

- Forgetting `.float()`: dividing a `uint8` tensor can give wrong types or zeros depending on the operation.
- `unsqueeze(0)` instead of `unsqueeze(1)`: you get $(1, N, 28, 28)$, one "image" with N channels.
- Looping over the 60 000 images in Python to filter them works, but takes ages. The mask does it in one operation.

Step 2. The DataLoaders

Goal of this step

Create `train_loader` and `test_loader`, which give the images in mini-batches of `BATCH_SIZE` = 128. The training set is shuffled, the test set is not.

What you need to know

- We never feed the whole dataset at once: we update the weights after every **mini-batch** of images. One pass over all mini-batches is an **epoch**.
- `TensorDataset(a, b, ...)` wraps tensors into a dataset; item `i` is the tuple `(a[i], b[i], ...)`. With a single tensor, item `i` is still a tuple, with one element.
- `DataLoader(dataset, batch_size=..., shuffle=...)` cuts a dataset into mini-batches and stacks them. Iterating over it gives one batch at a time.

Think first (answer on paper before coding)

1. With 5421 images and batches of 128, how many batches per epoch? How many images in the last one?

2. Why shuffle the training set at every epoch? Why is it pointless for the test set?
3. What will `for (x,) in train_loader:` give you, and why the strange comma?

Hint 1 (a nudge)

Two lines, one per loader. Each line wraps the tensor in a `TensorDataset`, then in a `DataLoader`.

Hint 2 (the tools)

`DataLoader(TensorDataset(x_train), batch_size=BATCH_SIZE, shuffle=True)`, and the same for the test set with the opposite shuffling.

Still stuck after both hints? Use rescue card 2 at the end of this booklet (page 15).

Checkpoint (you are on track if…)

`one batch: (128, 1, 28, 28) | batches per epoch: 43`. Indeed $5421 = 42 \times 128 + 45$: 42 full batches and a last one of 45 images.

Common pitfalls

- `for x in train_loader:` then `model(x)`: `x` is a *tuple*, and the model complains that it received a list. Unpack with `for (x,) in train_loader:`.

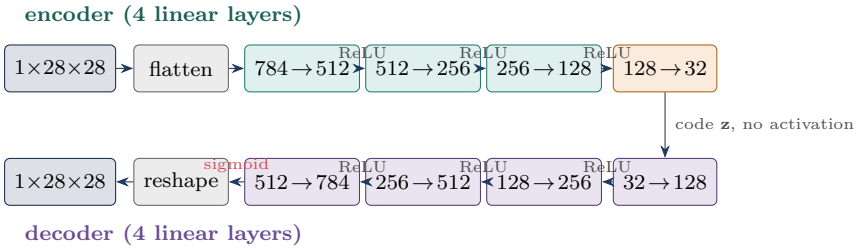
Step 3. The autoencoder

Goal of this step

Write the class `Autoencoder(latent_dim)` with two attributes, `self.encoder` and `self.decoder`:

- encoder: 4 linear layers, $784 \to 512 \to 256 \to 128 \to$ `latent_dim`;
- decoder: 4 linear layers, `latent_dim` $\to 128 \to 256 \to 512 \to 784$, with a **sigmoid** at the end.

The model takes images $(B, 1, 28, 28)$ and returns images $(B, 1, 28, 28)$.



What you need to know

- A PyTorch model is a class that inherits from `nn.Module`. Layers are created in `__init__` (after `super().__init__()`), and `forward(self, x)` says how data flows through them.
- `nn.Sequential(layer1, layer2, ...)` chains layers: the output of one is the input of the next.
- `nn.Linear(n_in, n_out)` expects vectors, i.e. tensors of shape (B, n_{in}). `nn.Flatten()` turns

$(B, 1, 28, 28)$ into $(B, 784)$; `nn.Unflatten(1, (1, 28, 28))` does the opposite.

- `nn.ReLU()` and `nn.Sigmoid()` are layers too: they go *inside* the `Sequential`, between the linear layers.

Think first (answer on paper before coding)

1. On the figure above, list in order every module of the encoder, then of the decoder (linear layers, activations, flatten, reshape). How many modules in each `Sequential`?
2. Why is there *no* activation after the last encoder layer?
3. Count the parameters of `nn.Linear(784, 512)`: one weight per connection, plus one bias per output. Do the same for the 7 other layers. What total do you expect? (Answer in Appendix B.)

Hint 1 (a nudge)

Build the encoder as one `nn.Sequential` and the decoder as another, by reading the figure from left to right and writing one module per box or arrow label. Then `forward` only has to pass `x` through the encoder, then through the decoder.

Hint 2 (the tools)

The encoder starts with `nn.Flatten()` and ends with `nn.Linear(128, latent_dim)`. The decoder ends with `nn.Linear(512, 784)`, `nn.Sigmoid()`, `nn.Unflatten(1, (1, 28, 28))`. Use `latent_dim`, not the number 32, so that the size can be changed later.

Still stuck after both hints? Use rescue card 3 at the end of this booklet (page 15).

Checkpoint (you are on track if…)

The check cell prints `input (128, 1, 28, 28) -> code (128, 32) -> output (128, 1, 28, 28)` and `1,141,296 parameters`. Before training, all output pixels are close to 0.5: a uniform grey image.

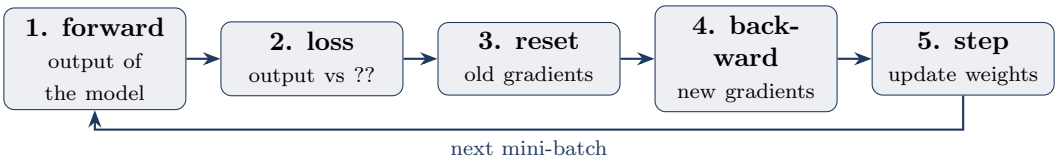
Common pitfalls

- `mat1 and mat2 shapes cannot be multiplied (3584x28 and 784x512)`: the image was not flattened before the first linear layer.
- Reshaping in `forward` instead of inside the decoder: the model works, but `model.decoder(z)` returns vectors, not images. The check cell tests both.
- Forgetting `super().__init__()`: PyTorch raises `cannot assign module before Module.__init__() call`.

Step 4. One epoch of training, and evaluation

Goal of this step

Write `train_one_epoch(model, loader, optimizer, loss_fn)`, which trains on every batch once and returns the **mean loss**, and `evaluate(model, loader, loss_fn)`, which only measures the mean loss.



What you need to know

- The five steps above are the same in *every* PyTorch training loop you will ever write. Learn their order by heart.
- `loss_fn = nn.MSELoss()` is created outside; calling `loss_fn(prediction, target)` returns a tensor with one number.
- `optimizer.zero_grad()`, `loss.backward()`, `optimizer.step()` are steps 3, 4, 5.
- `loss.item()` converts the one-number tensor to a Python float. Store floats, never tensors.
- `model.train()` and `model.eval()` switch the mode of the model (they matter for dropout and batch normalisation; we write them out of good habit). `@torch.no_grad()` above `evaluate` disables gradient tracking: faster, and no memory wasted.
- The batches are on the CPU; the model may be on the GPU. Move each batch with `x = x.to(device)`.

Think first (answer on paper before coding)

1. In step 2, what is the *target* of an autoencoder? Fill the "??" of the figure.
2. Which of the five steps does `evaluate` keep? Which must it absolutely not do?
3. To get the *mean* loss of the epoch, what do you accumulate inside the loop, and what do you divide by at the end?

Hint 1 (a nudge)

The loop is `for (x,) in loader:`. Inside: move `x` to the device, then the five steps, then add the loss of the batch to a running total. `evaluate` is the same loop with steps 1 and 2 only.

Hint 2 (the tools)

The loss line is `loss = loss_fn(model(x), x)`: the same `x` appears twice, once as input and once as target. The running total is `total += loss.item()`, and `len(loader)` is the number of batches.

Still stuck after both hints? Use rescue card 4 at the end of this booklet (page 15).

Checkpoint (you are on track if...)

Something close to `test loss before 0.2300 | train loss of the epoch 0.0974 | test loss after 0.0642`. Exact values change slightly from one machine to another; what matters is that the test loss drops a lot after one epoch.

Common pitfalls

- Forgetting `optimizer.zero_grad()`: gradients add up from batch to batch and training becomes erratic.
- `total += loss`: you keep the whole computation graph of every batch in memory. Use

`loss.item()`.

- `Expected all tensors to be on the same device:` the batch was not moved with `.to(device)`.

Step 5. Showing the reconstructions

Goal of this step

Write `show_reconstructions(model, images, title)`: run the model on a few images and draw two rows, the originals on top and their reconstructions below.

What you need to know

- This is `show_digits` with two rows instead of one: `plt.subplots(2, n, ...)` gives a 2-D array of axes, `axes[row, col]`.
- The images go to the model's device for the forward pass; the result comes back with `.cpu()` for matplotlib.
- We always display the *same* 8 test images, `fixed_images = x_test[:8]`, so that the figures of successive epochs can be compared.

Think first (answer on paper before coding)

Sketch the figure on paper: how many sub-plots, which tensor goes in which row?

Hint 1 (a nudge)

Three parts: compute `recon` for all the images at once; create a $2 \times n$ grid; loop over the columns and draw the original in row 0 and the reconstruction in row 1.

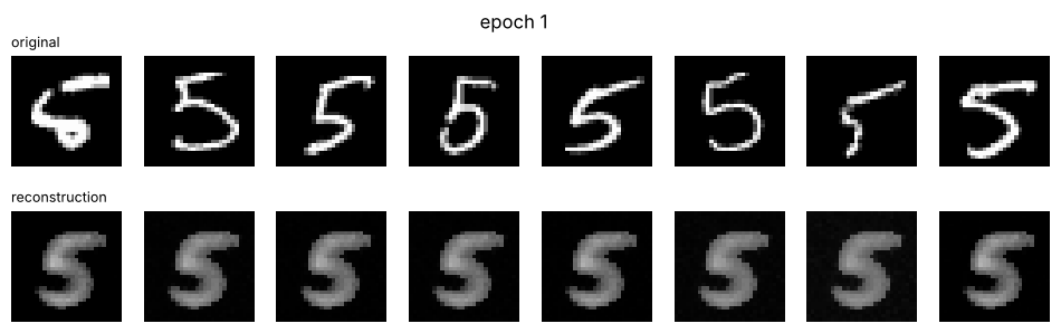
Hint 2 (the tools)

`recon = model(images.to(device)).cpu()`, then `axes[0, i].imshow(images[i, 0], ...)` and `axes[1, i].imshow(recon[i, 0], ...)`, with the same `cmap`, `vmin`, `vmax` as in Step 1. The decorator `@torch.no_grad()` is already above the function.

Still stuck after both hints? Use rescue card 5 at the end of this booklet (page 16).

Checkpoint (you are on track if...)

After only one epoch, the figure looks like this:



Do not panic: every reconstruction is the same blurry "average 5". This is normal. Early in training, the cheapest way to reduce the squared error for all images at once is to output the mean image. The differences between the 5s are learned during the next epochs.

Step 6. The full training loop

Goal of this step

Write `train(model, train_loader, test_loader, epochs, lr)`. It trains for `epochs` epochs and, after **every** epoch, stores the training and test losses in `history`, prints them, and shows the reconstructions of `fixed_images`. It returns `history`.

What you need to know

- The **loss function** and the **optimizer** are created *once*, before the loop over epochs: Adam keeps statistics about past gradients, which would be lost if you re-created it.
- `torch.optim.Adam(model.parameters(), lr=lr)` is a good default optimizer.
- `history` is a dictionary of two lists, `history["train"]` and `history["test"]`; `.append(value)` adds one value per epoch.
- All the building blocks already exist: you only assemble `train_one_epoch`, `evaluate` and `show_reconstructions`.

Think first (answer on paper before coding)

1. Write the function as a list of plain sentences first ("create…", "for each epoch…"). How many lines of code per sentence?
2. Why do we evaluate on the *test* set and not only look at the training loss?

Hint 1 (a nudge)

Two lines before the loop (loss function, optimizer). Inside `for epoch in range(1, epochs + 1):` one call to train, one call to evaluate, two appends, one print, one call to show the reconstructions.

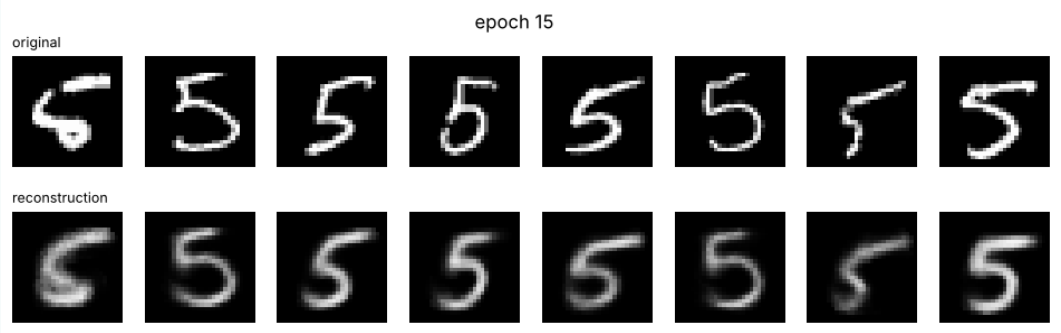
Hint 2 (the tools)

A readable print: `print(f"epoch {epoch}/{epochs} train loss {train_loss:.4f} test loss {test_loss:.4f}")`. Give the epoch number as the title of the figure, `title=f"epoch {epoch}"`, to know which figure is which.

Still stuck after both hints? Use rescue card 6 at the end of this booklet (page 16).

Checkpoint (you are on track if…)

With 15 epochs, the test loss goes from about 0.064 after epoch 1 to about 0.025 after epoch 15, and the last figure looks like this:



Each reconstruction now has the slant, size and loop of its own original. It is still blurrier than the input: 32 numbers cannot store every detail.

Common pitfalls

- Re-running the training cell continues to train the *same* model. To start from scratch, re-create the model (the cell does it for you).
- Creating the optimizer for one model and training another one: always pass `model.parameters()` of the model you train.

Step 7. The loss curves

Goal of this step

Write `plot_losses(history, title)`: one figure with the training and test losses against the epoch number (starting at 1), with axis labels, a legend and a title.

What you need to know

- `plt.plot(xs, ys, "o-", label="train")` draws a line with dots; call it once per curve.
- `plt.xlabel`, `plt.ylabel`, `plt.title`, `plt.legend()`, `plt.grid(alpha=0.3)`, then `plt.show()`.
- Python counts from 0, humans count epochs from 1: `range(1, len(history["train"]) + 1)`.

Hint 1 (a nudge)

Build the list of epoch numbers once, then plot two curves with the same x values and different labels.

Hint 2 (the tools)

`epochs = range(1, len(history["train"]) + 1)`, then `plt.plot(epochs, history["train"], "o-", label="train")` and the same for `"test"`.

Still stuck after both hints? Use rescue card 7 at the end of this booklet (page 16).

Checkpoint (you are on track if…)

Both curves decrease and stay very close to each other. A detail worth understanding: after epoch 1, the *training* loss (≈ 0.097) is *higher* than the test loss (≈ 0.064).

Think first (answer on paper before coding)

Questions for your report.

1. Why is the training loss of epoch 1 higher than the test loss? (Hint: *when* is each of them measured?)
2. Has training converged after 15 epochs? What would you try?
3. What would it mean if the test loss became much higher than the training loss?
4. Which details of the 5s are lost in the reconstructions, and why are the outputs blurry?

Step 8. From linear layers to convolutions

Goal of this step

Build `ConvAutoencoder(latent_dim)`: same idea, same 4 + 4 layers, same 32-number code, but with **convolutions** instead of linear layers. Then train it with your *unchanged* `train` function and compare it with the linear model.

8.1 What changes, what stays

	linear autoencoder	convolutional autoencoder
data, DataLoaders	*unchanged*	
`train_one_epoch`, `evaluate`, `train`, plots	*unchanged*	
input of the encoder	flattened vector $(B, 784)$	image $(B, 1, 28, 28)$: **no flatten**
encoder layers	`nn.Linear`	`nn.Conv2d`, stride 2 to shrink
decoder layers	`nn.Linear`	`nn.ConvTranspose2d`, stride 2 to grow
code **z**	$(B, 32)$	$(B, 32, 1, 1)$: 32 channels of size 1×1
last activation	sigmoid, *unchanged*	

This is the most important lesson of the step: **a well-organised training code does not care which model it trains.** Only one class changes.

8.2 The shape arithmetic

What you need to know

For a square input of size n, a layer with kernel size k, stride s and padding p produces a square output of size:

$$\underbrace{\left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1}_{\texttt{nn.Conv2d}} \qquad \underbrace{(n-1)\,s - 2p + k + op}_{\texttt{nn.ConvTranspose2d}}$$

where op is the `output_padding` argument (0 by default). The number of *channels* is simply the second argument of the layer: `nn.Conv2d(in_channels, out_channels, kernel_size, stride, padding)`.

Example. `nn.Conv2d(1, 16, 3, stride=2, padding=1)` on a $1 \times 28 \times 28$ image: $\lfloor (28 + 2 - 3)/2 \rfloor + 1 = \lfloor 13.5 \rfloor + 1 = 14$, so the output is $16 \times 14 \times 14$.

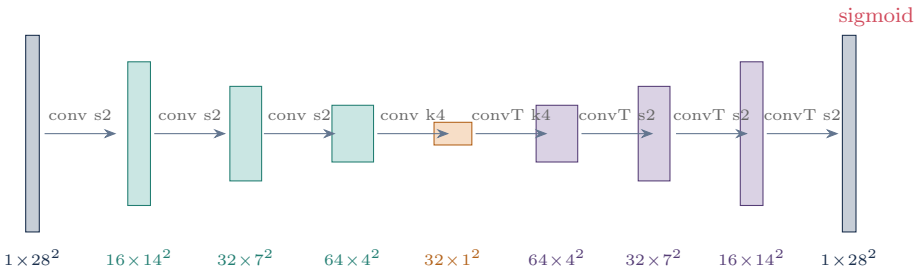
Think first (answer on paper before coding)

Fill in the table with the two formulas (answers in Appendix B). The encoder shrinks $28 \to 14 \to 7 \to 4 \to 1$; the decoder mirrors it.

layer	operation	output shape
input		$1 \times 28 \times 28$
enc 1	`Conv2d(1, 16, 3, stride=2, padding=1)` + ReLU	$16 \times 14 \times 14$
enc 2	`Conv2d(16, 32, 3, stride=2, padding=1)` + ReLU	_____
enc 3	`Conv2d(32, 64, 3, stride=2, padding=1)` + ReLU	_____
enc 4	`Conv2d(64, latent_dim, 4)` (no activation)	_____
dec 1	`ConvTranspose2d(latent_dim, 64, 4)` + ReLU	_____
dec 2	`ConvTranspose2d(64, 32, 3, stride=2, padding=1)` + ReLU	_____
dec 3	`ConvTranspose2d(32, 16, 3, 2, 1, output_padding=1)` + ReLU	_____
dec 4	`ConvTranspose2d(16, 1, 3, 2, 1, output_padding=1)` + Sigmoid	_____

(In the last two rows, `3, 2, 1` is short for `kernel_size=3, stride=2, padding=1`.) Then answer:

1. Why does `enc 4` use a kernel of size 4 and no padding? What does it do to a 4×4 map?
2. Why do `dec 3` and `dec 4` need `output_padding=1`, but not `dec 2`? (Compute `dec 2` with and without it.)



Hint 1 (a nudge)

Copy the structure of your linear `Autoencoder`: two `nn.Sequential`, one for the encoder and one for the decoder, and the same `forward`. Then translate each line of the table into one layer, followed by its activation.

Hint 2 (the tools)

- Drop `nn.Flatten()` and `nn.Unflatten(...)`: convolutions work directly on images.
- Keyword arguments make the code readable: `nn.Conv2d(16, 32, 3, stride=2, padding=1)`.
- Debug by printing: the check cell runs a batch through every layer and prints its output shape. Compare with your table.

Still stuck after both hints? Use rescue card 8 at the end of this booklet (page 16).

8.3 Train it and compare

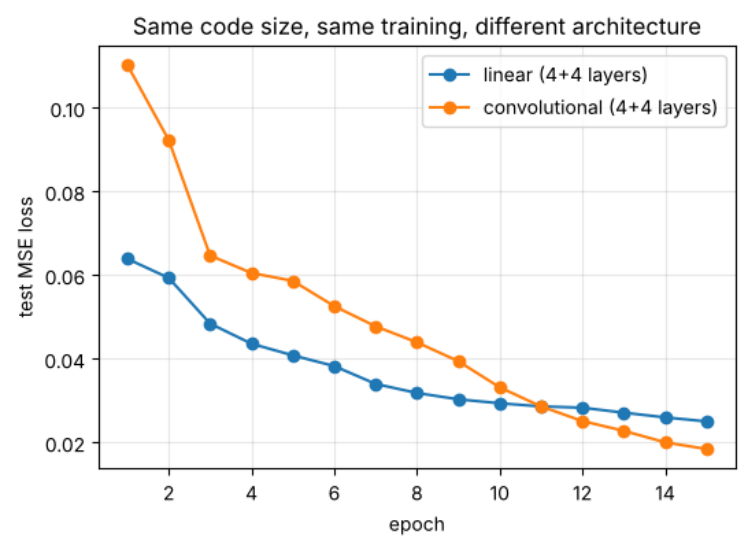
Think first (answer on paper before coding)

Before running: the convolutional model has about **10 times fewer** parameters than the linear one (112 161 versus 1 141 296). Do you expect it to reconstruct better or worse? Write down your bet.

Checkpoint (you are on track if…)

Train with exactly the same call as before: `train(conv_model, train_loader, test_loader, epochs=EPOCHS, lr=LEARNING_RATE)`. You should observe:

- a **slow start**: after epoch 1 the reconstructions are almost black, and the loss is higher than the linear model's;
- then a **faster improvement**: the curves cross around epoch 11, and after 15 epochs the convolutional model is ahead.



If you train both for 30 epochs, the convolutional model ends with about half the test loss of the linear one.

Think first (answer on paper before coding)

Questions for your report.

1. Why can a model with 10 times fewer parameters do better? Think about what a linear layer must learn to detect a curved stroke at every possible position, and what a convolution reuses.
2. Was your bet right?

Going further (optional)

- Change `LATENT_DIM` to 2, 8 and 64. How do the reconstructions change?
- Replace `nn.MSELoss()` by `nn.BCELoss()`: it treats each pixel as a probability, which is natural with a sigmoid output.
- Train on all ten digits (remove the filter in `load_digit`) and show reconstructions of every digit.
- Feed the model a 3 when it was trained only on 5s. What comes out, and why?

Appendix A. Rescue cards

Use a card only after both hints of its step. Each card is a skeleton: replace every `____` by the right expression. If a card still does not unblock you, ask a teacher before opening the solution notebook.

Rescue card 1. Load and display

```
def load_digit(digit=5, train=True):
    dataset = torchvision.datasets.MNIST(root="data", train=train, download=
        True)
    keep = ____                              # boolean mask on the labels
    images = dataset.data[____]              # (N, 28, 28), integers
    images = images.____() / ____            # floats in [0, 1]
    return images.unsqueeze(____)            # (N, 1, 28, 28)

def show_digits(images, n=8, title=""):
    fig, axes = plt.subplots(1, n, figsize=(1.5 * n, 1.8))
    for i in range(n):
        axes[i].imshow(images[____].detach().cpu(), cmap="gray", vmin=0, vmax
            =1)
        axes[i].axis("off")
    fig.suptitle(title)
    plt.show()
```

Rescue card 2. DataLoaders

```
train_loader = DataLoader(TensorDataset(____), batch_size=____, shuffle=____)
test_loader  = DataLoader(TensorDataset(____), batch_size=____, shuffle=____)
```

Rescue card 3. The model

```
self.encoder = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, ____), nn.ReLU(),
    nn.Linear(____, ____), nn.ReLU(),
    nn.Linear(____, ____), nn.ReLU(),
    nn.Linear(____, latent_dim),
)
self.decoder = nn.Sequential(
    nn.Linear(latent_dim, ____), nn.ReLU(),
    # ... two more Linear + ReLU ...
    nn.Linear(____, 784), ____,          # the activation that gives (0, 1)
    nn.Unflatten(1, (1, 28, 28)),
)

def forward(self, x):
    z = ____
    return ____
```

Rescue card 4. One epoch

```
for (x,) in loader:
    x = x.to(device)
    x_hat = ____                    # 1. forward
```

```
    loss = loss_fn(____, ____)    # 2. loss: prediction, target
    ____                          # 3. reset the gradients
    ____                          # 4. backward
    ____                          # 5. update the weights
    total += ____
# after the loop: return total / ____

# evaluate: the same loop, keeping only steps 1 and 2
```

Rescue card 5. Reconstructions

```
recon = model(____).cpu()
n = len(images)
fig, axes = plt.subplots(____, n, figsize=(1.5 * n, 3.3))
for i in range(n):
    axes[0, i].imshow(____, cmap="gray", vmin=0, vmax=1)
    axes[1, i].imshow(____, cmap="gray", vmin=0, vmax=1)
    axes[0, i].axis("off"); axes[1, i].axis("off")
fig.suptitle(title)
plt.show()
```

Rescue card 6. Training loop

```
loss_fn = ____
optimizer = torch.optim.Adam(____, lr=lr)
for epoch in range(1, epochs + 1):
    train_loss = train_one_epoch(____, ____, ____, ____)
    test_loss = evaluate(____, ____, ____)
    history["train"].append(____)
    history["test"].append(____)
    print(f"epoch {epoch}/{epochs}  train loss {train_loss:.4f}  test loss {
        test_loss:.4f}")
    show_reconstructions(____, ____, title=f"epoch {epoch}")
```

Rescue card 7. Loss curves

```
epochs = range(1, ____ + 1)
plt.figure(figsize=(6, 4))
plt.plot(epochs, ____, "o-", label="train")
plt.plot(epochs, ____, "o-", label="test")
plt.xlabel("epoch"); plt.ylabel("MSE loss"); plt.title(title)
plt.legend(); plt.grid(alpha=0.3); plt.show()
```

Rescue card 8. Convolutional autoencoder

```
self.encoder = nn.Sequential(
    nn.Conv2d(1, 16, 3, stride=2, padding=1), nn.ReLU(),       # 16x14x14
    nn.Conv2d(____, ____, 3, stride=2, padding=1), nn.ReLU(),  # 32x7x7
    nn.Conv2d(____, ____, 3, stride=2, padding=1), nn.ReLU(),  # 64x4x4
    nn.Conv2d(64, latent_dim, ____),                           # 32x1x1
)
self.decoder = nn.Sequential(
    nn.ConvTranspose2d(latent_dim, 64, ____), nn.ReLU(),           # 64x4x4
    nn.ConvTranspose2d(64, 32, 3, stride=2, padding=1), nn.ReLU(), # 32x7x7
    nn.ConvTranspose2d(32, 16, 3, stride=2, padding=1,
                       output_padding=____), nn.ReLU(),            # 16x14x14
```

```
    nn.ConvTranspose2d(16, 1, 3, stride=2, padding=1,
                       output_padding=____), ____,                    # 1x28x28
)
# forward: exactly the same as in the linear Autoencoder
```

Appendix B. Answers to the "Think first" calculations

The big picture

$784/32 = 24.5$: the image is compressed about 25 times. With a 784-number code, the network could learn to copy its input: zero error, nothing learned. The sigmoid matches the $[0, 1]$ range of the pixels; on the code it would squeeze all values into $(0, 1)$ for no reason and slow learning down.

Step 2

$5421 = 42 \times 128 + 45$: 43 batches, the last one with 45 images.

Step 3: counting parameters

A layer `nn.Linear(a, b)` has $a \times b$ weights and b biases.

encoder	parameters	decoder	parameters
$784 \to 512$	401 920	$32 \to 128$	4 224
$512 \to 256$	131 328	$128 \to 256$	33 024
$256 \to 128$	32 896	$256 \to 512$	131 584
$128 \to 32$	4 128	$512 \to 784$	402 192
total	570 272	total	571 024

Grand total: 1 141 296. Notice that the first and the last layer hold 70% of all parameters.

Step 8: the shape table

layer	computation	output
enc 1	$\lfloor(28 + 2 - 3)/2\rfloor + 1 = 14$	$16 \times 14 \times 14$
enc 2	$\lfloor(14 + 2 - 3)/2\rfloor + 1 = 7$	$32 \times 7 \times 7$
enc 3	$\lfloor(7 + 2 - 3)/2\rfloor + 1 = 4$	$64 \times 4 \times 4$
enc 4	$\lfloor(4 + 0 - 4)/1\rfloor + 1 = 1$	$32 \times 1 \times 1$
dec 1	$(1 - 1) \cdot 1 - 0 + 4 = 4$	$64 \times 4 \times 4$
dec 2	$(4 - 1) \cdot 2 - 2 + 3 = 7$	$32 \times 7 \times 7$
dec 3	$(7 - 1) \cdot 2 - 2 + 3 + 1 = 14$	$16 \times 14 \times 14$
dec 4	$(14 - 1) \cdot 2 - 2 + 3 + 1 = 28$	$1 \times 28 \times 28$

A kernel as large as the map (4×4, no padding) looks at the whole map at once and outputs a single value per channel: it plays the role of the last linear layer. `dec 2` must go from 4 to 7, an odd size: without `output_padding` it gives exactly 7, with it 8. `dec 3` and `dec 4` must exactly double ($7 \to 14$, $14 \to 28$): without `output_padding` they would give 13 and 25.

Appendix C. Error messages and what they mean

message (last line)	most likely cause
`mat1 and mat2 shapes cannot be multiplied`	input not flattened, or wrong `in_features` in a `Linear`
`Expected all tensors to be on the same device`	a batch or a tensor not moved with `.to(device)`
`Can't call numpy() on Tensor that requires grad`	plotting a model output: add `.detach()` or use `torch.no_grad()`
`Invalid shape (1, 28, 28) for image data`	passing `images[i]` to `imshow` instead of `images[i, 0]`
`'list' object has no attribute 'to'`	forgot to unpack the batch: `for (x,) in loader:`
warning `Using a target size that is different to the input size`	output and input shapes differ: check the decoder's last layers
`NotImplementedError`	a cell of the starter notebook is still to be written
loss does not decrease	missing `optimizer.step()`, `zero_grad()` called between `backward()` and `step()`, or wrong target